

Sacolas

Nome do arquivo: `sacolas.c`, `sacolas.cpp`, `sacolas.pas`, `sacolas.java`, `sacolas.js` ou `sacolas.py`

Marcelo, um entusiasta do churrasco, decidiu que o próximo fim de semana seria palco de seu "Churrasco Lendário". Com uma lista de convidados que crescia a cada hora, ele se dirigiu ao seu açougue de confiança com uma missão: comprar as melhores carnes da cidade.

Empolgado, Marcelo não se conteve. Comprou picanha, maminha, linguiça toscana, coxa de frango, e até umas costeletas de porco. No total, foram N pacotes de carne, numerados de 1 a N , com o i -ésimo pacote tendo peso P_i .

O problema começou quando ele chegou ao caixa. Ele percebeu que o açougue cobrava por cada sacola usada, e portanto ele deseja minimizar o número de sacolas usadas. Porém, ele viu que cada sacola suporta no máximo peso S . Acima disso, a sacola pode rasgar.

Agora, Marcelo se vê diante de um dilema. Ele precisa levar todos os N pacotes de carne para casa, mas para agilizar o empacotamento, precisa organizar os pacotes em intervalos consecutivos, da forma como foram passados no caixa. Ou seja, cada sacola deve conter um intervalo contíguo de pacotes de carne. Mais precisamente:

- Para cada sacola, deve existir um par de inteiros (l, r) com $1 \leq l \leq r \leq N$ tal que a sacola contém os pacotes $l, l+1, \dots, r$.
- A soma dos pesos em cada sacola deve ser no máximo S , ou seja, $P_l + P_{l+1} + \dots + P_r \leq S$.
- Todo pacote deve estar em alguma sacola.
- Nenhum pacote deve estar em mais de uma sacola.

Ajude Marcelo a descobrir qual o número mínimo de sacolas que ele precisará usar, garantindo que a soma dos pesos em cada sacola não ultrapasse S .

Entrada

A primeira linha da entrada contém dois inteiros, N e S , que indicam o número total de pacotes de carne e o limite de peso que uma sacola suporta, respectivamente.

A segunda linha da entrada contém N inteiros, $P_1, P_2, \dots, P_N$, que representam os pesos de cada pacote.

Saída

Seu programa deve imprimir uma única linha contendo um único inteiro, o número mínimo de sacolas que Marcelo precisa comprar.

Restrições

- $1 \leq N \leq 100\,000$.
- $1 \leq S \leq 1\,000\,000\,000$.
- $1 \leq P_i \leq S$ para todo $1 \leq i \leq N$.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (25 pontos):** $P_i = 1$ para todo $1 \leq i \leq N$.
- **Subtarefa 3 (25 pontos):** É garantido que Marcelo precisa de no máximo 3 sacolas.
- **Subtarefa 4 (25 pontos):** $N \leq 10$.
- **Subtarefa 5 (25 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
6 6 2 3 3 1 1 2	3

Explicação do exemplo 1: Nesse caso o número mínimo de sacolas é 3. Podemos dividir os pacotes nas sacolas da seguinte maneira:

- A primeira sacola fica com o intervalo (1, 1), note que $2 \leq 6$.
- A segunda sacola fica com o intervalo (2, 3), note que $3 + 3 \leq 6$.
- A terceira sacola fica com o intervalo (4, 6), note que $1 + 1 + 2 \leq 6$.

Perceba que esta não é a única forma de dividir os pacotes nas sacolas. Note também que não podemos dividir em 2 sacolas da seguinte forma: $3 + 3 \leq 6$ e $2 + 1 + 1 + 2 \leq 6$ pois, apesar das sacolas suportarem os pesos, esta divisão não organiza os pacotes em intervalos consecutivos.

Exemplo de entrada 2	Exemplo de saída 2
7 3 1 1 1 1 1 1 1	3

Exemplo de entrada 3	Exemplo de saída 3
8 10 6 3 4 5 2 7 3 5	4

Fotos de Relíquias

Nome do arquivo: `fotos.c`, `fotos.cpp`, `fotos.pas`, `fotos.java`, `fotos.js` ou `fotos.py`

Jonas, um famoso arqueólogo, quer viralizar nas redes sociais publicando fotos de relíquias encontradas em suas jornadas. Para tirar fotos agradáveis, Jonas escolheu uma paisagem como plano de fundo para suas fotos.

A paisagem escolhida por Jonas é dividida em N seções, de forma que algumas são seções de destaque e outras não. Ele escreveu uma lista A de N inteiros que descrevem as N seções da paisagem, da esquerda para a direita, de forma que, se $A_i = 1$ então a i -ésima seção é de destaque, e caso contrário $A_i = 0$.

Jonas escolheu uma relíquia e vai tirar uma foto dela usando uma região contínua da paisagem como plano de fundo. Para que a relíquia seja o foco principal, a foto deve conter uma única seção de destaque, onde a relíquia será posicionada. Portanto, ele deve escolher um par de inteiros (l, r) que satisfaça as seguintes condições:

- $1 \leq l \leq r \leq N$.
- As seções $l, l + 1, \dots, r$ farão parte da foto.
- Dentre as seções presentes na foto, exatamente uma é de destaque, ou seja, existe um único índice k tal que $l \leq k \leq r$ e $A_k = 1$.

Jonas quer saber de quantas maneiras diferentes ele pode tirar a foto. Ajude ele a determinar a quantidade de pares de inteiros (l, r) que satisfazem todas as condições desejadas.

Entrada

A primeira linha da entrada contém um único inteiro N , indicando o número de seções na paisagem.

A segunda linha da entrada possui N inteiros $A_1, A_2, \dots, A_N$, onde $A_i = 1$ se a i -ésima seção é uma seção de destaque, ou $A_i = 0$ caso contrário.

Saída

Seu programa deve imprimir uma única linha contendo um único inteiro, o número de modos que Jonas pode tirar a foto.

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 100\,000$.
- $A_i = 0$ ou $A_i = 1$ para todo $1 \leq i \leq N$.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). (*Competidores usando Python ou JavaScript podem ignorar este aviso.*)

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (20 pontos):** $N \leq 100$.
- **Subtarefa 3 (22 pontos):** $N \leq 3000$.
- **Subtarefa 4 (26 pontos):** Existe exatamente uma seção de destaque na paisagem, ou seja, há um único índice i na lista tal que $A_i = 1$.
- **Subtarefa 6 (32 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
5 1 0 0 1 0	9

Explicação do exemplo 1: Os pares (l, r) que seguem as condições para a foto são: $(1, 1)$, $(1, 2)$, $(1, 3)$, $(2, 4)$, $(2, 5)$, $(3, 4)$, $(3, 5)$, $(4, 4)$ e $(4, 5)$.

Exemplo de entrada 2	Exemplo de saída 2
12 0 0 0 1 0 0 0 0 0 0 0 0	36

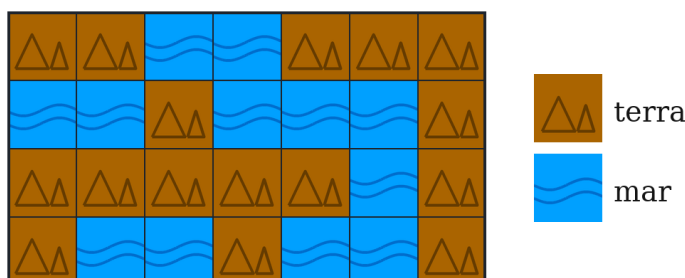
Explicação do exemplo 2: Este exemplo satisfaz as restrições da subtarefa 4.

Fitas Verde-amarelas

Nome do arquivo: `fitas.c`, `fitas.cpp`, `fitas.pas`, `fitas.java`, `fitas.js` ou `fitas.py`

Com a ajuda dos alunos de Ciência da Computação, o Instituto de Geociências da Unicamp descobriu a existência de um arquipélago (conjunto de ilhas) no Oceano Atlântico. Para homenagear os alunos, o arquipélago foi chamado de Nlogônia. Agora, a professora Ada embarcou em uma viagem para visitar a Nlogônia pela primeira vez.

Ciente de que seu celular pode não ter sinal nas ilhas, Ada está levando um mapa que ela mesma desenhou. Este mapa consiste de uma malha quadriculada (*grid*) com N linhas e M colunas, onde cada célula pode ser terra ou mar. A figura abaixo ilustra um exemplo de mapa.

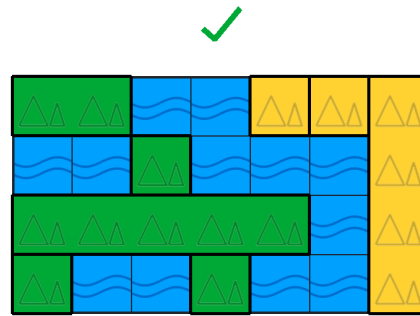


Com as turbulências do voo, o mapa acabou sendo danificado com rasgos e furos. Ada possui dois rolos de fita adesiva semi-transparente, um de fita verde e um de fita amarela, dos quais ela vai cortar fitas para consertar o mapa. Cada fita pode ser usada para cobrir um segmento contíguo (horizontal ou vertical) de células. Ao colar uma fita, Ada precisa alinhar a fita com as linhas da malha, de modo que cada célula esteja completamente coberta pela fita, ou completamente descoberta.

Ada consegue cortar fitas de qualquer comprimento que desejar, inclusive de comprimento 1 (que cobrem apenas uma célula). Além disso, ela pode cortar quantas fitas desejar de cada rolo, e confirmou que cada um dos rolos possui fita suficiente para cobrir o mapa inteiro se necessário. Porém, considerando a utilidade do mapa e o senso estético de Ada, ela estabeleceu as seguintes regras para consertar o mapa:

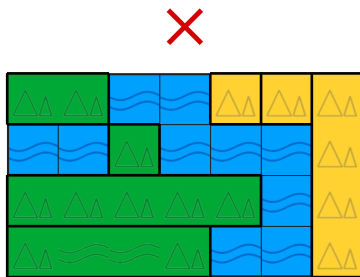
- Todas as células de terra devem estar cobertas por fitas.
- Todas as células de mar devem estar descobertas.
- Fitas verdes só podem ser coladas na horizontal. Ou seja, uma fita verde pode cobrir qualquer segmento contíguo de células na mesma linha.
- Fitas amarelas só podem ser coladas na vertical. Ou seja, uma fita amarela pode cobrir qualquer segmento contíguo de células na mesma coluna.
- Uma célula coberta com uma fita amarela **não** pode ser vizinha de uma célula coberta com uma fita verde. Duas células são vizinhas se elas compartilham um lado (ou seja, consideramos apenas vizinhas horizontais e verticais, não diagonais).

A figura abaixo ilustra um modo válido de consertar o mapa mostrado acima usando 8 fitas, sendo 5 verdes e 3 amarelas. Observe que é permitido que células vizinhas na diagonal sejam cobertas com a mesma cor.

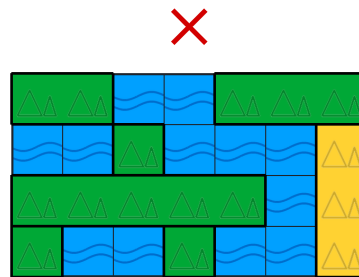


Configuração válida.

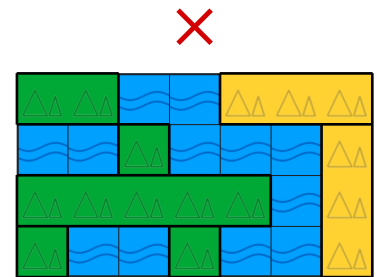
As figuras abaixo ilustram três modos inválidos de consertar o mapa usando 7 fitas. Em cada uma delas, alguma regra não é respeitada.



Existem células de mar que estão cobertas.



Existe uma fita verde adjacente a uma fita amarela.



Existe uma fita amarela colada na horizontal.

Apesar de toda a sua genialidade, a professora Ada tem dificuldade com o processo de cortar fitas dos rolos (ela reclama que as fitas colam nas mãos antes de ela usar a tesoura). Por isso, ela pediu sua ajuda. Ajude Ada a determinar o número **mínimo de fitas** que ela precisa colar no mapa para cobrir o mapa de maneira válida, ou seja, respeitando todas as regras acima.

Entrada

A primeira linha da entrada contém dois inteiros N e M , o número de linhas e o número de colunas do mapa, respectivamente.

As próximas N linhas possuem M caracteres cada (sem espaços em branco) e descrevem o mapa. O j -ésimo caractere da i -ésima linha, c_{ij} , é ‘#’ (sem aspas) caso a célula (i, j) seja de terra, ou ‘.’ (sem aspas) caso a célula (i, j) seja de mar.

Saída

Seu programa deve imprimir uma única linha contendo um único inteiro, o número mínimo de fitas que Ada precisa colar para consertar o mapa de acordo com as regras que ela estabeleceu.

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 1\,000$.
- $1 \leq M \leq 1\,000$.
- $c_{ij} = \text{‘#’}$ ou $c_{ij} = \text{‘.’}$ para todos $1 \leq i \leq N$ e $1 \leq j \leq M$.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas restrições adicionais às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta sub tarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (10 pontos):** $N = 2$ e $M = 2$.
- **Subtarefa 3 (19 pontos):** $N = 1$.
- **Subtarefa 4 (12 pontos):**
 - $N \geq 2$ e $M \geq 2$.
 - Existe apenas uma célula de mar.
- **Subtarefa 5 (20 pontos):** As células de terra formam um retângulo sólido (formalmente, existem duas células (l_1, c_1) e (l_2, c_2) tais que uma célula (i, j) possui terra se, e somente se, $l_1 \leq i \leq l_2$ e $c_1 \leq j \leq c_2$).
- **Subtarefa 6 (19 pontos):**
 - $N \leq 100$ e $M \leq 100$.
 - Em cada linha e em cada coluna, as células de terra formam um segmento contíguo (ou seja, não existem dois segmentos disjuntos de terra na mesma linha ou na mesma coluna).
- **Subtarefa 7 (20 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
<pre>4 7 ##..### ..#...# #####. #..#...#</pre>	<pre>8</pre>

Explicação do exemplo 1: Este é o exemplo mostrado no enunciado. O enunciado mostra um modo de cobrir o mapa usando oito fitas, e é possível provar que não há como cobrir o mapa de maneira válida usando sete ou menos fitas.

Exemplo de entrada 2	Exemplo de saída 2
<pre>1 9 ...#####</pre>	<pre>2</pre>

Explicação do exemplo 2: Este exemplo satisfaz as restrições da sub tarefa 3. Podemos usar duas fitas verdes para cobrir o mapa corretamente.

Exemplo de entrada 3	Exemplo de saída 3
<pre> 6 5### ..### ..### ..### </pre>	<pre> 3 </pre>

Explicação do exemplo 3: Este exemplo satisfaz as restrições da subtarefa 5.

Exemplo de entrada 4	Exemplo de saída 4
<pre> 8 9##... ..####... ...###...##.##### ##..... </pre>	<pre> 7 </pre>

Explicação do exemplo 4: Este exemplo satisfaz as restrições da subtarefa 6.

Exemplo de entrada 5	Exemplo de saída 5
<pre> 5 15 ####.####.### #..#..#..#...#. #..#..####...#. #..#..#..#...#. ####.####.### </pre>	<pre> 18 </pre>

Empregos de Júlio

Nome do arquivo: `empregos.c`, `empregos.cpp`, `empregos.pas`, `empregos.java`, `empregos.js` ou `empregos.py`

Júlio trabalha muito para sustentar sua família. No entanto, ele percebeu que um único emprego não seria suficiente para pagar as contas e, por isso, decidiu arranjar dois! Assim, ele foi contratado para ser entregador de jornais e segurança de shopping. A princípio, ele pensou que conseguiria trabalhar em ambos ao mesmo tempo, mas logo descobriu que isso não seria tão fácil.

Como os dois serviços demandam muito tempo e esforço, Júlio desistiu de pegar turnos em ambos ao mesmo tempo. Dessa forma, ele pretende escolher em qual deles ele vai trabalhar em cada um dos próximos N dias, sendo que, para cada um desses, Júlio sabe o quanto ele ganharia se trabalhasse como entregador ou segurança. Mais especificamente, no i -ésimo dia, ele ganhará a_i reais para entregar jornais e b_i reais para vigiar o shopping.

O chefe de Júlio no emprego de segurança ficou sabendo da decisão de seu empregado e, a fim de segurá-lo no trabalho, ofereceu um incentivo a ele. Assim, ele prometeu a Júlio que, após trabalhar K dias na segurança do shopping, ele ganhará em dobro em todo e qualquer turno que ele escolher, ou seja, ganhará $2b_i$ reais se vigiar o shopping no i -ésimo dia. Note que os primeiros K dias trabalhados nesse emprego não precisam ser consecutivos, e ele ganhará o valor normal do dia.

Júlio está muito interessado na proposta do chefe, mas, como a quantidades N e K de dias são muito grandes, ele não sabe em qual emprego ele deve trabalhar em cada dia. Sua tarefa é: dados os valores N e K , bem como as listas $a_1 \dots a_N$ e $b_1 \dots b_N$, ajude Júlio a saber qual o **máximo** de dinheiro que ele pode conseguir, considerando a proposta feita por um de seus chefes.

Entrada

A primeira linha da entrada contém dois inteiros, N e K , que indicam o número de dias que Júlio irá trabalhar e a quantidade de vezes que ele deve trabalhar de segurança para ganhar em dobro.

A segunda linha da entrada contém N inteiros, $a_1, a_2, \dots, a_N$, que representam o quanto ele receberia em cada dia caso trabalhasse entregando jornais.

A terceira linha da entrada contém N inteiros, $b_1, b_2, \dots, b_N$, que representam o quanto ele receberia em cada dia caso trabalhasse vigiando o shopping.

Saída

A saída deve conter uma única linha contendo um inteiro, o máximo que Júlio consegue receber nos próximos N dias.

Restrições

- $1 \leq N \leq 100\,000$.
- $0 \leq K \leq N$.
- $1 \leq a_i, b_i \leq 1\,000\,000\,000$, para todo $1 \leq i \leq N$.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). (*Competidores usando Python ou JavaScript podem ignorar este aviso.*)

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (13 pontos):** $K = 0$ e $N \leq 1\,000$.
- **Subtarefa 3 (21 pontos):** $K = 1$ e $N \leq 1\,000$.
- **Subtarefa 4 (31 pontos):** $N \leq 1\,000$.
- **Subtarefa 5 (14 pontos):** $b_i = b_j$, para todo $1 \leq i \leq j \leq N$.
- **Subtarefa 6 (21 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
5 0 30 40 30 50 70 10 30 20 20 40	260

Explicação do exemplo 1: Este exemplo satisfaz as restrições da subtarefa 2. Note que, como $K = 0$, Júlio ganha em dobro no segundo emprego desde o início. Assim, ele pode trabalhar como entregador nos dias 1 e 4, e como segurança nos dias 2, 3 e 5. Dessa forma, ele ganharia $30 + 2 \cdot 30 + 2 \cdot 20 + 50 + 2 \cdot 40 = 260$ reais.

Exemplo de entrada 2	Exemplo de saída 2
5 1 30 40 30 50 70 10 30 20 20 40	240

Explicação do exemplo 2: Este exemplo satisfaz as restrições da subtarefa 3. Note que, como $K = 1$, Júlio ganha em dobro no segundo emprego após o primeiro dia de trabalho como segurança. Assim, ele pode trabalhar como entregador no dia 4 e como segurança nos dias 1, 2, 3 e 5. Dessa forma, ele ganharia $10 + 2 \cdot 30 + 2 \cdot 20 + 50 + 2 \cdot 40 = 240$ reais.

Exemplo de entrada 3	Exemplo de saída 3
5 2 30 40 25 15 20 10 10 10 10 10	130

Explicação do exemplo 3: Este exemplo satisfaz as restrições da subtarefa 5.

Redes de Descanso

Nome do arquivo: `redes.c`, `redes.cpp`, `redes.pas`, `redes.java`, `redes.js` ou `redes.py`

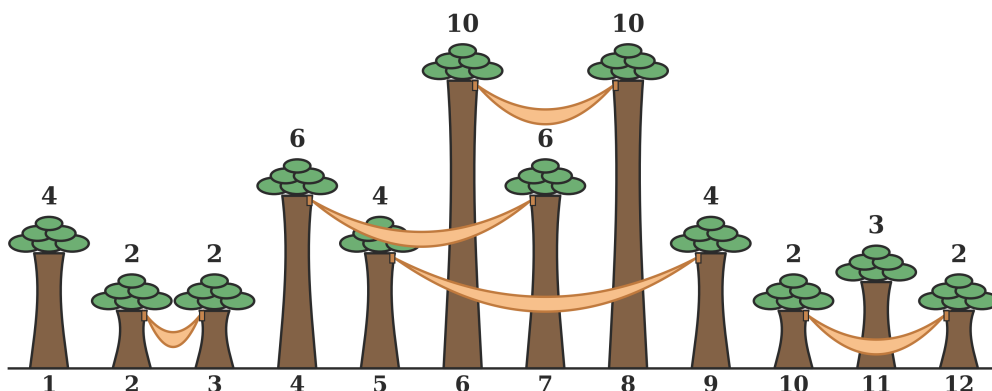
O departamento de Antropologia da Unicamp tem estudado os guaranis, o maior povo indígena do Brasil. Como parte de uma viagem do departamento, você está visitando uma tribo que possui o hábito de dormir em redes de descanso penduradas em árvores.

Na região da floresta perto da aldeia, existem N árvores alinhadas, numeradas de 1 a N da esquerda para a direita, com alturas variadas. Por motivos de segurança, o cacique da aldeia determinou algumas regras para pendurar redes:

- Cada rede deve estar pendurada em duas árvores distintas que possuam a mesma altura.
- Em cada árvore, no máximo uma rede pode ser pendurada.
- Duas redes penduradas em árvores de mesma altura não devem se intersectar – ou seja, se duas redes estão penduradas na mesma altura, então uma delas precisa estar completamente à esquerda da outra.

Observe que sempre é possível pendurar uma rede em duas árvores de mesma altura que não tenham nenhuma rede pendurada ainda, mesmo que entre as duas existam árvores mais altas (a rede pode passar pela frente das árvores altas).

A figura abaixo ilustra um exemplo com 12 árvores, com a altura de cada árvore indicada acima dela, e um modo válido de pendurar 5 redes:



A aldeia está se preparando para uma grande festa. Ao fim desta festa, todos os convidados dormirão em redes. O cacique precisa saber quantas pessoas ele pode convidar para festa, sabendo que somente uma pessoa pode dormir em cada rede. Deste modo, sua tarefa é ajudar o cacique a descobrir qual o número **máximo** de redes que ele consegue pendurar nas árvores, respeitando todas as regras que ele estabeleceu.

Entrada

A primeira linha da entrada contém um único inteiro N , o número de árvores alinhadas na floresta.

A segunda linha da entrada contém N inteiros $a_1, a_2, \dots, a_N$, separados por espaços em branco, indicando as alturas das N árvores da esquerda para a direita.

Saída

Seu programa deverá imprimir uma única linha contendo um único inteiro, o número máximo de redes que podem ser penduradas nas árvores.

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 100\,000$.
- $1 \leq a_i \leq 100\,000$ para todo $1 \leq i \leq N$.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (20 pontos):** $a_i = 1$ para todo $1 \leq i \leq N$.
- **Subtarefa 3 (20 pontos):** $a_i \leq 3$ para todo $1 \leq i \leq N$.
- **Subtarefa 4 (20 pontos):**
 - $N \leq 100$.
 - $a_i \leq 100$ para todo $1 \leq i \leq N$.
 - Não existem três árvores com a mesma altura.
- **Subtarefa 5 (20 pontos):**
 - $N \leq 100$.
 - $a_i \leq 100$ para todo $1 \leq i \leq N$.
- **Subtarefa 6 (20 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
12 4 2 2 6 4 10 6 10 4 2 3 2	5

Explicação do exemplo 1: Este é o exemplo mostrado no enunciado. A figura do enunciado ilustra um modo de pendurar 5 redes. É possível verificar que é impossível pendurar 6 redes.

Exemplo de entrada 2	Exemplo de saída 2
7 1 1 1 1 1 1 1	3

Exemplo de entrada 3	Exemplo de saída 3
10 7 2 1 3 2 3 5 5 7 6	4

Explicação do exemplo 3: Este exemplo satisfaz as restrições da subtarefa 4.

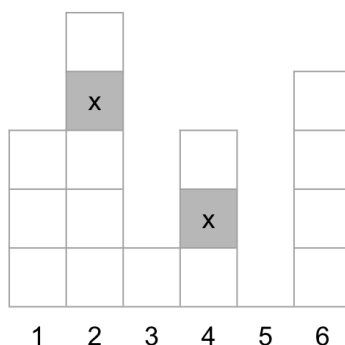
Diagonal

Nome do arquivo: `diagonal.c`, `diagonal.cpp`, `diagonal.pas`, `diagonal.java`, `diagonal.js` ou `diagonal.py`

Joana é dona de uma construtora muito famosa. Em seu novo empreendimento, ela está construindo um condomínio de N prédios, onde atualmente o i -ésimo prédio possui A_i apartamentos, um por andar. Como o condomínio ainda está em construção, é possível que a quantidade de apartamentos em um prédio ainda seja 0. Mas nos prédios onde há pelo menos um apartamento, eles são numerados a partir da base, ou seja, o apartamento no primeiro andar recebe o número 1, o apartamento no segundo andar recebe o número 2, e assim por diante, até o apartamento no último andar que recebe o número A_i .

Joana é uma entusiasta da construção civil, por isso está sempre usando as técnicas mais avançadas neste setor. Dessa vez, ela construiu o condomínio utilizando um material especial, que prometia ser mais resistente. Porém, foram descobertos problemas em relação à propagação de som neste material. Um funcionário identificou que, quando alguém em um determinado apartamento emite um som, este som é ouvido em todos os apartamentos que estão em algum prédio à esquerda, em um andar mais alto e que estejam na mesma diagonal que o apartamento inicial.

Mais precisamente, se alguém no apartamento i do prédio j emite um som, ele é ouvido no apartamento a do prédio b se e somente se $b < j$ e $a - i = j - b$. Por exemplo, na figura a seguir, se um som é emitido no apartamento 2 do prédio 4, ele será ouvido no apartamento 4 do prédio 2, mas não será ouvido no apartamento 1 do prédio 3 (pois o apartamento não está acima do andar inicial) nem no apartamento 4 do prédio 6 (pois o prédio não está à esquerda do prédio inicial). A figura abaixo ilustra esse cenário.



Joana ficou muito decepcionada ao saber desse problema. Porém, quantos apartamentos ouvem um som depende de onde ele é inicialmente emitido. Assim, para avaliar os possíveis danos, Joana deseja que você calcule o **maior número de apartamentos** (incluindo o apartamento inicial) que podem ouvir um som emitido em algum apartamento do condomínio.

Entrada

A primeira linha da entrada contém um único inteiro N , representando a quantidade de prédios do condomínio.

A segunda linha da entrada contém N inteiros, $A_1, A_2, \dots, A_N$, que representam a quantidade de apartamentos de cada prédio, da esquerda para a direita.

Saída

Seu programa deverá produzir uma única linha contendo somente um inteiro, o maior número de apartamentos que podem ouvir um som emitido em algum apartamento do condomínio, se o apartamento inicial for escolhido de maneira a maximizar esta quantidade.

Restrições

- $1 \leq N \leq 300\,000$.
- $0 \leq A_i \leq 1\,000\,000\,000$ para todo $1 \leq i \leq N$.

Informações sobre a pontuação

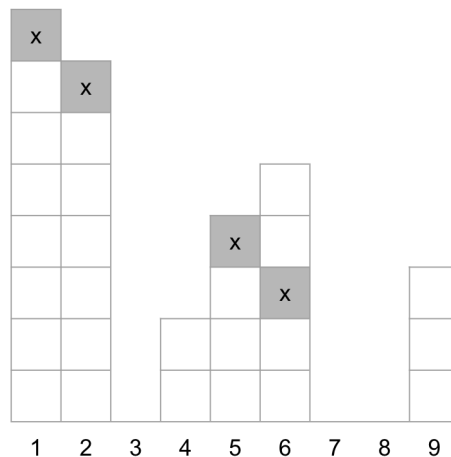
A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (29 pontos):** $N \leq 1\,000$ e $A_i \leq 1\,000$ para todo $1 \leq i \leq N$.
- **Subtarefa 3 (41 pontos):** $N \leq 1\,000$.
- **Subtarefa 4 (30 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
9 8 7 0 2 4 5 0 0 3	4

Explicação do exemplo 1: O número máximo de apartamentos é 4, que é atingido se escolhermos como apartamento inicial o apartamento 3 do prédio 6, conforme ilustrado na figura abaixo.



É possível verificar que não existe nenhuma escolha de apartamento inicial que faria com que 5 ou mais apartamentos ouvissem o som.

Exemplo de entrada 2	Exemplo de saída 2
10 1 16 27 5 4 27 381 29 57 1000	7

Exemplo de entrada 3	Exemplo de saída 3
1 0	0

Recarga

Nome do arquivo: `recaga.c`, `recaga.cpp`, `recaga.pas`, `recaga.java`, `recaga.js` ou `recaga.py`

Jonas quer fazer uma viagem de Pelém para Pique-Pique, duas grandes cidades do Prasil, seu país de origem, mas seu carro quebrou! Sendo assim, ele decide comprar um carro elétrico.

Na hora de escolher qual carro comprar, ele percebe que cada um tem uma bateria de **capacidade** diferente. Essa decisão pode ter um grande impacto, uma vez que nem todas as cidades tem uma **estação de recarga** para a bateria, e portanto uma capacidade baixa pode tornar a viagem inviável.

Sabemos que existem N cidades no Prasil, sendo Pelém a cidade 1 e Pique-Pique a cidade N . Ademais, existem M rodovias de mão dupla, onde a i -ésima rodovia liga a cidade a_i com a cidade b_i . Ao percorrer a i -ésima rodovia, Jonas sabe que um carro elétrico consome c_i unidades de energia da bateria. Note que a bateria não pode ter energia negativa em nenhum momento, e portanto, Jonas só pode percorrer a rodovia i se a bateria tiver ao menos c_i unidades de energia.

Também existem K cidades com estações de recarga para carros elétricos. Ao chegar em uma dessas cidades, Jonas pode recuperar a energia da bateria por completo, ou seja, a energia da bateria volta a ser a capacidade máxima. É garantido que Pelém (a cidade 1) tem uma estação de recarga, e Pique-Pique (a cidade N) não tem.

Dadas essas informações, ajude Jonas respondendo qual é a menor capacidade para a bateria de seu carro elétrico de forma que seja possível fazer uma viagem de Pelém até Pique-Pique. Perceba que não é necessário minimizar a distância total percorrida, nem a quantidade de cidades intermediárias, Jonas está interessado apenas na menor capacidade para a bateria com a qual é possível realizar a viagem (isso porque ele notou que, quanto maior a capacidade da bateria, o carro é mais caro).

Entrada

A primeira linha da entrada contém dois inteiros, N e M , que indica o número total de cidades e o número total de rodovias, respectivamente.

As próximas M linhas possuem três inteiros cada e descrevem as rodovias. A i -ésima destas linhas contém três inteiros a_i , b_i e c_i indicando que as cidades a_i e b_i são ligadas pela i -ésima rodovia, e a quantidade de energia c_i consumida para percorrê-la.

A próxima linha tem um inteiro K , a quantidade de cidades com estações de recarga.

A última linha contém K inteiros $x_1, x_2, \dots, x_K$, indicando as cidades que contém uma estação de recarga.

Saída

A saída deve conter uma única linha contendo um inteiro, a capacidade mínima que a bateria do carro de Jonas precisa ter para ele conseguir fazer sua viagem de Pelém para Pique-Pique.

Restrições

- $2 \leq N \leq 50\,000$.
- $1 \leq M \leq 100\,000$.
- $1 \leq a_i, b_i \leq N$ e $a_i \neq b_i$, para todo $1 \leq i \leq M$.
- É garantido que não há duas rodovias diferentes que liguem o mesmo par de cidades.

- $1 \leq c_i \leq 10^9$, para todo $1 \leq i \leq M$.
- $1 \leq K < N$
- $1 \leq x_j < N$, para todo $1 \leq j \leq K$.
- A cidade 1 contém uma estação de recarga e a cidade N não contém.
- É garantido que com uma capacidade de bateria suficientemente alta é possível fazer uma viagem entre qualquer par de cidades.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). *(Competidores usando Python ou JavaScript podem ignorar este aviso.)*

Informações sobre a pontuação

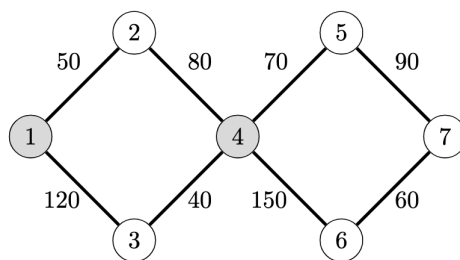
A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (22 pontos):** $N, M \leq 1000$ e $K = 1$, ou seja, apenas a cidade 1 possui estação de recarga.
- **Subtarefa 3 (23 pontos):** $K = N - 1$, ou seja, todas as cidades exceto N possuem estação de recarga.
- **Subtarefa 4 (29 pontos):** $N, M \leq 1000$ e $K \leq 100$.
- **Subtarefa 5 (26 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
7 8 1 2 50 1 3 120 2 4 80 3 4 40 4 5 70 4 6 150 5 7 90 6 7 60 2 1 4	160

Explicação do exemplo 1: Neste exemplo temos 7 cidades e 8 rodovias conforme a figura a seguir:



Perceba que, com uma capacidade de 160 unidades de energia, Jonas consegue realizar a viagem da cidade 1 para a cidade 7 da seguinte forma:

- Jonas começa sua viagem na cidade 1 com 160 unidades de energia.
- Jonas viaja da cidade 1 para a cidade 3, o que consome 120 unidades de energia, restando em sua bateria $160 - 120 = 40$.
- Jonas viaja da cidade 3 para a cidade 4, o que consome 40 unidades de energia, restando em sua bateria $40 - 40 = 0$.
- Na cidade 4 há uma estação de recarga, portanto, Jonas recarrega sua bateria, que volta a ter 160 unidades de energia.
- Jonas viaja da cidade 4 para a cidade 5, o que consome 70 unidades de energia, restando em sua bateria $160 - 70 = 90$.
- Jonas viaja da cidade 5 para a cidade 7, o que consome 90 unidades de energia, restando em sua bateria $90 - 90 = 0$.
- Jonas finalmente chega na cidade 7, conforme desejado. Perceba que, neste exemplo de viagem ele chega com 0 unidade de energia em sua bateria, mas isso não importa para Jonas.

É possível verificar que com uma capacidade menor que 160 é impossível viajar da cidade 1 para a cidade 7.

Exemplo de entrada 2	Exemplo de saída 2
5 6 1 2 10 1 3 60 2 4 30 3 5 70 4 5 90 2 5 100 3 1 3 4	70

Escadaria

Nome do arquivo: `escadaria.c`, `escadaria.cpp`, `escadaria.pas`, `escadaria.java`,
`escadaria.js` ou `escadaria.py`

O arqueólogo Jonas encontrou uma antiga escadaria nas ruínas do arquipélago da Nlogônia.

Jonas sabe que originalmente cada degrau tinha uma altura inteira e também que a escadaria seguia uma regra especial: a diferença entre as alturas de dois degraus consecutivos nunca poderia ser maior que 1. Por exemplo, se um degrau possuía altura 5, o próximo poderia ter altura 4, 5 ou 6, mas não poderia ter altura 2, 3 ou 7, pois a diferença seria maior do que 1.

O problema é que parte da escadaria foi destruída. Em algumas posições a altura original ainda é conhecida, mas em outras ela é completamente desconhecida. No total existem N degraus, e Jonas escreveu um número A_i associado ao i -ésimo degrau:

- Se $A_i = -1$, então a altura do degrau i é desconhecido.
- Se $A_i > 0$, então a altura do degrau i é A_i .

Sua tarefa é ajudar Jonas a terminar sua pesquisa sobre as ruínas de Nlogônia. Para isso, ele precisa saber, para cada um dos N degraus, a **maior** altura possível que o degrau pode ter em alguma reconstrução válida.

Entrada

A primeira linha da entrada possui o inteiro N , indicando a quantidade de degraus da escadaria.

A segunda linha possui N inteiros $A_1, A_2, \dots, A_N$, onde o i -ésimo inteiro indica o valor anotado por Jonas para o degrau i . Se $A_i = -1$, a altura do degrau i é desconhecida; caso contrário, a altura do degrau i é A_i .

Saída

Seu programa deve imprimir uma única linha contendo N inteiros (separados por espaços em branco), onde o i -ésimo deles representa a maior altura possível do degrau i em alguma reconstrução válida da escadaria.

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $2 \leq N \leq 200\,000$.
- $-1 \leq A_i \leq 1\,000\,000$.
- $A_i \neq 0$.
- É garantido que existe ao menos uma maneira válida de completar a escadaria.
- Existe pelo menos um degrau com altura conhecida inicialmente (ou seja, algum i tal que $A_i \neq -1$).

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (7 pontos):**
 - $N = 3$.
 - $A_1 \neq -1$, $A_2 = -1$ e $A_3 \neq -1$. Ou seja, somente A_2 é desconhecido.
- **Subtarefa 3 (22 pontos):** $N \leq 1000$ e só existe um i tal que $A_i \neq -1$. Ou seja, só um degrau tem sua altura conhecida.
- **Subtarefa 4 (15 pontos):** $N \leq 1000$.
- **Subtarefa 5 (25 pontos):** Apenas A_1 e A_N são diferentes de -1 . Ou seja, os únicos degraus com altura conhecida são 1 e N .
- **Subtarefa 6 (31 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
5 5 -1 -1 -1 6	5 6 7 7 6

Explicação do exemplo 1: 5, 6, 7, 7, 6 são os maiores valores possíveis para as alturas de cada um dos degraus de 1 até 5, respectivamente.

Por exemplo, o maior valor possível para o degrau 1 é 5, pois a altura deste degrau já é conhecida; e o maior valor possível para o degrau 3 é 7, pois, de todas sequências válidas de alturas, esse é o maior valor que o degrau 3 pode ter.

Note que a sequência 5, 4, 5, 6, 6 é válida, mas não corresponde aos valores máximos dos índices de 2 a 4.

Exemplo de entrada 2	Exemplo de saída 2
12 -1 -1 4 -1 -1 -1 2 -1 -1 2 -1 -1	6 5 4 5 4 3 2 3 3 2 3 4

Exemplo de entrada 3	Exemplo de saída 3
10 -1 -1 -1 10 -1 -1 -1 -1 -1 -1	13 12 11 10 11 12 13 14 15 16

Hidroviás e Rodovias

Nome do arquivo: `hidrovias.c`, `hidrovias.cpp`, `hidrovias.pas`, `hidrovias.java`,
`hidrovias.js` ou `hidrovias.py`

O arqueólogo Jonas continuou sua pesquisa no arquipélago da Nlogônia e subiu a escadaria, evitando os degraus destruídos. No topo das ruínas, ele encontrou uma grande sala, a qual continha diversas relíquias feitas de um metal desconhecido, bem como um mapa com um X marcado. O arqueólogo ficou muito animado com a descoberta, mas achou o sistema de transportes da região muito complicado.

O arquipélago da Nlogônia possui N ilhas, as quais são interligadas por M conexões, cada uma conectando duas ilhas distintas. Cada conexão pode ser percorrida nas duas direções e pode ser uma rodovia ou uma hidrovia. Jonas sabe que um caminho entre duas ilhas A e B é uma sequência $A = i_1, i_2, \dots, i_k = B$ de ilhas distintas em que existe alguma conexão, de qualquer tipo, entre todo par de ilhas consecutivas. Observe que é possível que o sistema de transportes **não** seja conectado, ou seja, é possível que não exista qualquer caminho entre algum par de ilhas.

Além disso, o arqueólogo estudou muito a História da região e sabe que, nos últimos K anos:

- Nenhuma hidrovia foi construída.
- No máximo uma rodovia foi construída em cada ano. Ou seja, **no máximo K rodovias** foram construídas ao todo.

Sabendo que o arquipélago era um local muito próspero no passado, Jonas se perguntou como as pessoas não se perdiam nos diversos caminhos, e imaginou que o sistema de transportes deveria ser simples há K anos atrás. Ele define que uma rede de transportes é simples se **existe no máximo um caminho entre qualquer par de ilhas no mapa**.

Portanto, dada a atual rede de transportes do arquipélago, ajude Jonas a verificar se é possível que ela fosse simples há K anos atrás. Isto é, determine se existe um sistema em que há no máximo um caminho entre qualquer par de ilhas e que condiz com a História da região descrita pelo arqueólogo.

Entrada

A primeira linha da entrada possui três inteiros N , M e K indicando, respectivamente, o número de ilhas no arquipélago da Nlogônia, o número de conexões entre pares de ilhas, e a quantidade de anos considerada por Jonas.

As próximas M linhas possuem três inteiros cada e descrevem as conexões. A i -ésima destas linhas contém três inteiros a_i , b_i e t_i indicando as ilhas a_i e b_i que são ligadas pela i -ésima conexão. t_i representa o tipo da conexão: $t_i = 1$ indica que essa conexão é uma hidrovia, e $t_i = 2$ indica que essa conexão é uma rodovia.

É garantido que não existem duas conexões que ligam o mesmo par de ilhas. Além disso, nenhuma conexão liga uma ilha a ela mesma.

Saída

Seu programa deverá imprimir uma única linha contendo um único caractere. Caso exista um sistema de transportes simples que seja condizente com a História do arquipélago, imprima o caractere **S** (a letra S maiúscula). Caso não exista, imprima o caractere **N** (a letra N maiúscula).

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 100\,000$.
- $1 \leq M \leq 100\,000$.
- $0 \leq K \leq 100\,000$.
- $1 \leq a_i, b_i \leq N$ e $a_i \neq b_i$ para todo $1 \leq i \leq M$.
- $1 \leq t_i \leq 2$ para todo $1 \leq i \leq M$.
- Não existem duas conexões que ligam o mesmo par de ilhas.

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

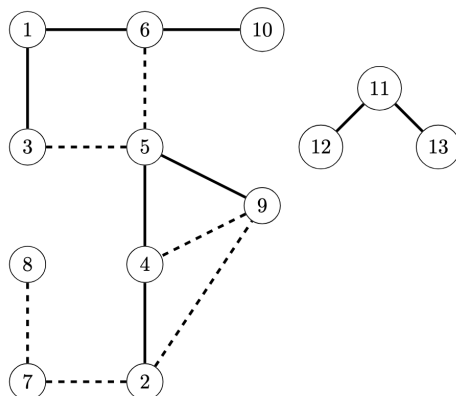
- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (11 pontos):** Todas as conexões são hidrovias.
- **Subtarefa 3 (11 pontos):** Todas as conexões são rodovias. Além disso, para quaisquer duas ilhas u e v , existe pelo menos um caminho de u para v .
- **Subtarefa 4 (14 pontos):** Todas as conexões são rodovias.
- **Subtarefa 5 (18 pontos):** $K = 1$.
- **Subtarefa 6 (19 pontos):** $N \leq 1\,000$ e $M \leq 1\,000$.
- **Subtarefa 7 (27 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
13 14 3 2 9 2 4 2 1 7 8 2 4 5 1 1 3 1 5 3 2 9 4 2 7 2 2 6 1 1 5 9 1 5 6 2 10 6 1 11 12 1 11 13 1	S

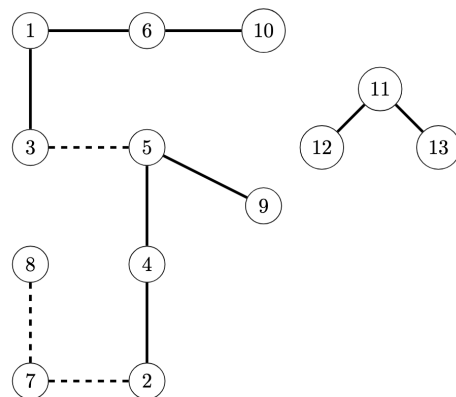
Explicação do exemplo 1:

Atualmente, o sistema de transportes da Nlogônia pode ser visto da seguinte forma:



Note que as linhas contínuas indicam as hidrovias e as linhas tracejadas indicam rodovias. No entanto, nos últimos 3 anos, é possível que as rodovias (2,9), (9,4) e (5,6) tenham sido construídas.

Portanto, a seguinte rede de transportes é condizente com os conhecimentos do arqueólogo:



Exemplo de entrada 2	Exemplo de saída 2
10 9 2 8 2 1 2 3 1 3 4 1 4 5 1 5 6 1 6 7 1 2 5 1 2 6 1 1 10 1	N

Explicação do exemplo 2: Este exemplo satisfaz as restrições da subtarefa 2.

Note que o atual arquipélago da Nlogônia não possui rodovias, logo, o sistema local de transportes manteve-se inalterado nos últimos 2 anos.

No entanto, existem dois caminhos entre as ilhas 5 e 6: $5 - 6$ e $5 - 2 - 6$.

Logo, é impossível que a rede de transportes tenha sido simples há 2 anos.

Exemplo de entrada 3	Exemplo de saída 3
6 9 3 1 2 2 5 3 2 5 2 2 2 3 2 1 6 2 1 4 2 6 2 2 5 1 2 4 5 2	N

Explicação do exemplo 3: Este exemplo satisfaz as restrições das subtarefas 3 e 4.

Harmonia Nasal

Nome do arquivo: `nasal.c`, `nasal.cpp`, `nasal.pas`, `nasal.java`, `nasal.js` ou `nasal.py`

O arqueólogo Jonas deseja chegar no local marcado com um X em seu mapa, mas, para isso, precisa saber ler as legendas escritas nele. Jonas descobriu que o texto está escrito no idioma do povo Guarani¹, e por isso precisa aprender a língua.

Em particular, ele deve se tornar proficiente no fenômeno da **harmonia nasal**, em que um som nasal (por exemplo ‘ã’) pode afetar como outras partes da palavra são pronunciadas.

Para isso, ele anotou N sílabas presentes na língua Guarani, sendo que a i -ésima sílaba tem **nasalidade** a_i e **resistência** b_i . Ele quer formar uma única palavra de maior nasalidade possível usando o seguinte processo:

1. Primeiro, Jonas pode zerar a resistência de no máximo K sílabas, ou seja, ele pode escolher até K sílabas e transformar b_i em 0.
2. Depois, Jonas escolhe uma única sílaba i para começar a palavra. Sendo assim, a palavra começará com nasalidade $A = a_i$ e resistência $B = b_i$. Vale ressaltar que durante todo o processo existirá uma única palavra, sua nasalidade será denominada A e sua resistência B . Observe que a sílaba escolhida i nunca mais pode ser usada.
3. Por fim, ele pode realizar a seguinte operação inúmeras vezes (inclusive nenhuma vez), desde que cumpra todas as condições:
 - Jonas pode escolher uma sílaba não escolhida ainda i tal que $a_i + A \geq B$ e anexá-la ao fim da palavra.
 - A palavra passa a ter nasalidade $a_i + A - B$ e resistência b_i .
 - Vale ressaltar que a sílaba escolhida i nunca mais pode ser usada.

Dada a lista de sílabas e o fator K , ajude Jonas dizendo qual é a maior nasalidade de uma palavra que ele consegue obter usando o processo descrito acima.

Entrada

A primeira linha da entrada contém dois inteiros, N e K , que indicam o número de sílabas que Jonas está trabalhando e o número de vezes que ele pode fazer a primeira operação.

A segunda linha da entrada contém N inteiros, $a_1, a_2, \dots, a_N$, que representam o fator de nasalidade de cada uma das sílabas.

A terceira linha da entrada contém N inteiros, $b_1, b_2, \dots, b_N$, que representam o fator de resistência de cada uma das sílabas.

Saída

A saída deve conter uma única linha contendo um inteiro, a maior nasalidade que Jonas consegue obter.

Restrições

- $1 \leq N \leq 100\,000$.
- $0 \leq K \leq N$.

¹Em particular, o povo Guarani-Mbyá, que vive no sul e sudeste do Brasil.

- $1 \leq a_i, b_i \leq 1\,000\,000\,000$, para todo $1 \leq i \leq N$.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). (*Competidores usando Python ou JavaScript podem ignorar este aviso.*)

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (12 pontos):** $K = N - 1$.
- **Subtarefa 3 (11 pontos):** $b_i = b_{i+1}$ para todo $1 \leq i < N$. Em outras palavras, todas as resistências são iguais.
- **Subtarefa 4 (21 pontos):** $a_i \geq b_i$ para todo $1 \leq i \leq N$.
- **Subtarefa 5 (25 pontos):** $K = 0$.
- **Subtarefa 6 (12 pontos):** $K \leq 10$.
- **Subtarefa 7 (19 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
5 1 3 5 2 1 7 9 2 3 1 5	13

Explicação do exemplo 1: Nesse caso, Jonas pode obter uma palavra de nasalidade 13 da seguinte forma:

- Jonas primeiro zera b_1 . Agora a sílaba 1 tem resistência 0.
- Jonas começa sua palavra com a sílaba 1. Agora ela tem nasalidade $A = 3$ e resistência $B = 0$.
- Jonas adiciona a sílaba 2, com $a_2 = 5$ e $b_2 = 2$. Perceba que ele pode realizar essa operação pois a sílaba 2 ainda não foi escolhida e $a_2 + A \geq B$. Agora a palavra passa a ter nasalidade $a_2 + A - B = 5 + 3 - 0 = 8$ e resistência $b_2 = 2$.
- Jonas adiciona a sílaba 5, com $a_5 = 7$ e $b_5 = 5$. Perceba que ele pode realizar essa operação pois a sílaba 5 ainda não foi escolhida e $a_5 + A \geq B$ (nesse momento a palavra tem nasalidade $A = 8$ e resistência $B = 2$). Então a palavra passa a ter nasalidade $a_5 + A - B = 7 + 8 - 2 = 13$ e resistência $b_5 = 5$.

Note que Jonas poderia adicionar mais sílabas, mas para maximizar a nasalidade ele escolheu não fazer isso, nesse caso.

Exemplo de entrada 2	Exemplo de saída 2
5 2 1 2 3 4 5 2 2 2 2 2	12

Explicação do exemplo 2: Esse exemplo satisfaz a subtarefa 3.

Exemplo de entrada 3	Exemplo de saída 3
1 0 10 10	10

Explicação do exemplo 3: Nesse caso é apenas possível formar uma palavra de nasalidade 10 e resistência 10. Esse exemplo satisfaz a subtarefa 2.

Exemplo de entrada 4	Exemplo de saída 4
1 1 10 10	10

Explicação do exemplo 4: Note que apesar de ser possível mudar a resistência da sílaba, a resposta não muda em relação ao exemplo 3.

Forja de ORicalco

Nome do arquivo: `forja.c`, `forja.cpp`, `forja.pas`, `forja.java`, `forja.js` ou `forja.py`

O arqueólogo Jonas finalmente chegou no local marcado com um X, o grande vulcão ORiginário, responsável pela formação das ilhas do Arquipélago da Nlogônia. Como Jonas aprendeu o idioma local, ele conseguiu ler uma anotação de seu mapa, que contava a lenda do ORicalco, uma liga metálica muito valiosa antigamente produzida no vulcão. Nos tempos antigos, esse metal gerou riqueza e prosperidade, mas também trouxe guerras e destruição para a região. A lenda também mencionava algum tipo de maldição do lugar, mas o arqueólogo estava mais interessado na parte que explicava a confecção do ORicalco.

Jonas descobriu que existe uma forja secreta dentro do vulcão e que, para criar ORicalco, é necessário fundir diversas pepitas de metal nela. Para isso, o arqueólogo deve posicionar em linha os N metais que utilizará no processo, os quais têm graus de impureza $a_1, a_2, \dots, a_N$. Em seguida, ele pode escolher uma sequência de **exatamente K pepitas consecutivas**, fundi-las em um único metal e colocá-lo de volta **na mesma posição da linha em que as K originais estavam**. Esse processo pode ser repetido enquanto existem ao menos K pepitas na forja.

A lenda explica que, ao fundir as pepitas $i, i+1, \dots, i+K-1$, o metal resultante tem impureza $a_i | a_{i+1} | \dots | a_{i+K-1}$, em que $|$ representa a operação binária OR. Esta operação é realizada em cada bit da seguinte maneira (sendo x e y dois bits):

x	0	0	1	1
y	0	1	0	1
x y	0	1	1	1

Ou seja, o bit resultante da operação OR será 0 apenas quando os dois bits forem 0, senão, será 1.

Por exemplo, se $K = 2$, $a_1 = 10$ e $a_2 = 12$, a forja criaria uma pepita de impureza $a_1 | a_2 = 10 | 12 = 1010_2 | 1100_2 = 1110_2 = 14$, conforme:

10	1	0	1	0
12	1	1	0	0
$10 12 = 14$	1	1	1	0

Vale ressaltar que, nas linguagens de programação, o operador OR binário é simplesmente $|$, portanto, sendo a e b dois inteiros, $a|b$ é o resultado do OR binário entre eles.

Jonas pode fundir os metais quantas vezes quiser (inclusive, nenhuma vez) e, por fim, deve reunir **todas** as pepitas restantes e compactá-las, formando uma barra de ORicalco. **O grau de impureza do ORicalco equivale à soma das impurezas das pepitas que restarem na forja.**

O arqueólogo deseja produzir o metal de menor impureza possível, porém não é muito bom em operações binárias. Por isso, dadas as quantidades N e K , bem como as impurezas $a_1, a_2, \dots, a_N$ dos metais, ajude Jonas a calcular a **menor impureza possível** da barra de ORicalco. Note que os metais já estão fixados na forja e não podem ser trocados de lugar.

Entrada

A primeira linha da entrada contém dois inteiros, N e K , que indicam o número de metais e o tamanho dos conjuntos que podem ser fundidos.

A segunda linha da entrada contém N inteiros, $a_1, a_2, \dots, a_N$, que representam as impurezas dos metais, na ordem em que estão dispostos na forja.

Saída

A saída deve conter uma única linha contendo um inteiro, a menor impureza de ORicalco que Jonas consegue atingir.

Restrições

- $1 \leq N \leq 100\,000$.
- $2 \leq K \leq 100\,000$.
- $0 \leq a_i < 2^{30}$, para todo $1 \leq i \leq N$.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). (*Competidores usando Python ou JavaScript podem ignorar este aviso.*)

Informações sobre a pontuação

A tarefa vale 100 pontos. Estes pontos estão distribuídos em subtarefas, cada uma com suas **restrições adicionais** às definidas acima.

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (9 pontos):** $K = 2$ e $N \leq 100$.
- **Subtarefa 3 (12 pontos):** $K = 3$ e $N \leq 100$.
- **Subtarefa 4 (15 pontos):** $N \leq 100$.
- **Subtarefa 5 (17 pontos):** $N \leq 3\,000$.
- **Subtarefa 6 (13 pontos):** $a_i \leq 3$, para todo $1 \leq i \leq N$.
- **Subtarefa 7 (34 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
5 2 16 10 6 1 1	31

Explicação do exemplo 1: Nesse exemplo, Jonas pode forjar um ORicalco com grau de impureza 31 da seguinte forma:

- Jonas primeiro funde os metais no intervalo $[4, 5]$: $a_4|a_5 = 1|1 = 1_2|1_2 = 1_2 = 1$. A forja passa a conter: 16 10 6 1.
- Em seguida, Jonas funde os metais no intervalo $[2, 3]$: $a_2|a_3 = 10|6 = 1010_2|0110_2 = 1110_2 = 14$. Agora, a forja contém: 16 14 1.
- Por fim, ele compacta as pepitas restantes, produzindo uma barra de ORicalco de impureza $16 + 14 + 1 = 31$

Note que esse exemplo condiz com a Subtarefa 2.

Exemplo de entrada 2	Exemplo de saída 2
8 3 6 2 1 2 2 4 1 5	12

Explicação do exemplo 2: Nesse exemplo, Jonas pode forjar um ORicalco com grau de impureza 12 da seguinte forma:

- Jonas primeiro funde os metais no intervalo $[3, 5]$: $a_3|a_4|a_5 = 1|2|2 = 01_2|10_2|10_2 = 11_2 = 3$. A forja passa a conter: 6 2 3 4 1 5.
- Em seguida, Jonas funde o intervalo $[1, 3]$: $a_1|a_2|a_3 = 6|2|3 = 110_2|010_2|011_2 = 111_2 = 7$. Agora, a forja contém: 7 4 1 5.
- Enfim, ele funde as pepitas em $[4, 6]$: $a_4|a_5|a_6 = 4|1|5 = 100_2|001_2|101_2 = 101_2 = 5$. A forja passa a conter: 7 5.
- Por fim, ele compacta as pepitas restantes, produzindo uma barra de ORicalco de impureza $7 + 5 = 12$

Note que esse exemplo condiz com a Subtarefa 3.

Exemplo de entrada 3	Exemplo de saída 3
11 5 136 896 777 130 128 0 128 960 65 655 524	1635

Exemplo de entrada 4	Exemplo de saída 4
11 4 2 0 2 0 1 1 2 1 3 0 3	5

Explicação do exemplo 4: Note que esse exemplo condiz com a Subtarefa 6.

Energia

Nome do arquivo: `energia.c`, `energia.cpp`, `energia.pas`, `energia.java`, `energia.js` ou `energia.py`

Quando o arqueólogo Jonas terminou de forjar a barra de ORicalco, uma antiga maldição do vulcão foi imediatamente ativada. O chão tremeu, e diante dele surgiu novamente uma imensa escadaria, formada por N degraus, onde cada degrau i ($1 \leq i \leq N$) tinha uma altura representada por A_i .

Nesse momento, Jonas lembrou de um conceito importante das suas aulas de física vulcânica: a energia gasta para ir de um degrau i até um degrau j depende apenas das posições e das alturas dos dois degraus, assim como uma diferença de potencial entre dois pontos. A fórmula usada para calcular a energia gasta para ir de um degrau i para o j é:

$$E(i, j) = |i - j| + |A_i - A_j|$$

Onde $|x|$ representa o valor absoluto de x .

Sendo um mestre dos vulcões, Jonas sabe que um par (i, j) , com $1 \leq i < j \leq N$, é chamado de vulcônico se $E(i, j) = K$. Ele também decidiu ler a parte da lenda que explicava que, para se livrar da maldição, Jonas precisaria falar a todo momento quantos pares (i, j) são vulcônicos.

Para a surpresa do pesquisador, a partir do minuto 1, os degraus vão pouco a pouco desmoronando. No minuto t , o degrau D_t desmorona, fazendo com que seja impossível ir de qualquer degrau a esquerda de D_t para algum a direita de D_t .

No total, Q degraus desmoronaram nos minutos $1, 2, \dots, Q$. Então Jonas precisa falar, para cada minuto t de 0 (o minuto inicial, quando nenhum degrau havia desmoronado) até Q , quantos pares (i, j) existem tal que $E(i, j) = K$ e nenhum degrau desmorou entre (i, j) ainda. Ajude Jonas a escapar dessa terrível maldição!

Entrada

A primeira linha da entrada contém três inteiros N , Q e K , indicando respectivamente o número de degraus, o número de desmoronamentos e o valor alvo da energia.

A segunda linha contém N inteiros $A_1, A_2, \dots, A_N$, descrevendo a altura de cada degrau.

Em seguida, seguem Q linhas, onde a t -ésima delas contém um inteiro D_t , representando o degrau que desmoronou no minuto t .

Saída

O programa deve imprimir $Q + 1$ linhas. A primeira linha deve conter o número de pares vulcônicos existentes antes de qualquer desmoronamento. Cada uma das Q linhas seguintes deve conter o número de pares vulcônicos após cada um dos desmoronamentos.

Restrições

É garantido que todo caso de teste satisfaz as restrições abaixo.

- $1 \leq N \leq 200\,000$.
- $0 \leq Q < N$.
- $1 \leq A_i, K \leq 1\,000\,000\,000$.

Para competidores que utilizam C++ ou Java: Observe que alguns valores na saída podem ser muito grandes para caberem em um inteiro de 32 bits. É recomendado o uso de inteiros de 64 bits (`long long` em C++; `long` em Java). (*Competidores usando Python ou JavaScript podem ignorar este aviso.*)

Informações sobre a pontuação

- **Subtarefa 1 (0 pontos):** Esta subtarefa é composta apenas pelos exemplos mostrados abaixo. Ela não vale pontos, serve apenas para que você verifique se o seu programa imprime o resultado correto para os exemplos.
- **Subtarefa 2 (10 pontos):** $Q = 0$, $N \leq 3\,000$.
- **Subtarefa 3 (19 pontos):** $N \leq 3\,000$.
- **Subtarefa 4 (31 pontos):** $Q = 0$.
- **Subtarefa 5 (35 pontos):** $N \leq 60\,000$.
- **Subtarefa 6 (5 pontos):** Sem restrições adicionais.

Exemplos

Exemplo de entrada 1	Exemplo de saída 1
8 3 5 2 9 4 7 1 8 9 8 6 2 1	6 2 1 1

Explicação do exemplo 1: Inicialmente, os pares (i, j) com energia igual a 5 são: $(1, 5), (2, 6), (2, 7), (3, 5), (4, 7), (4, 8)$. $E(3, 5) = |3 - 5| + |4 - 1| = 5$, por exemplo. Depois de bloquear a posição 6, os pares são: $(1, 5), (3, 5)$. Depois de bloquear a posição 2, o único par é: $(3, 5)$. Depois de bloquear a posição 1, o único par é: $(3, 5)$.

Exemplo de entrada 2	Exemplo de saída 2
7 0 10 10 23 8 15 16 13 14	6

Explicação do exemplo 2: Os pares (i, j) com energia igual a 10 são: $(1, 5), (1, 7), (3, 5), (3, 7), (2, 4), (2, 5)$. $E(1, 7) = |1 - 7| + |10 - 14| = 10$, por exemplo.